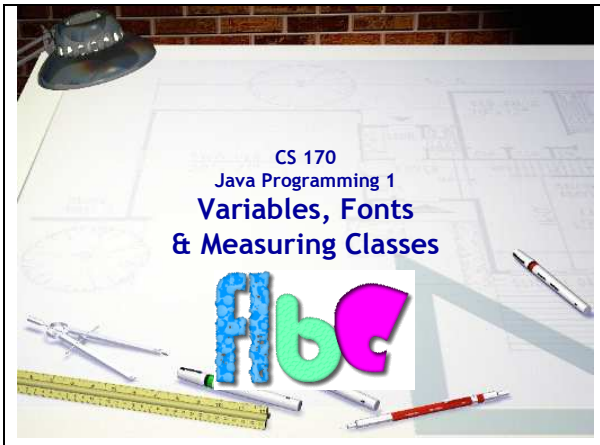



## Slide 1

**CS 170**
**Java Programming 1**
**Variables, Fonts**
**& Measuring Classes**
Duration: 00:00:16
Advance mode: Auto

## Notes:

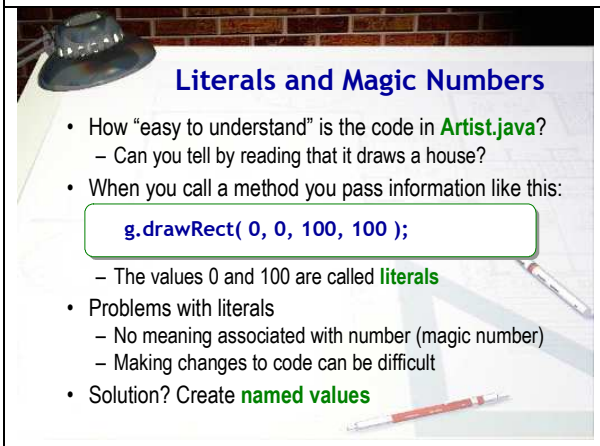
When we left off last week (just before the midterm exam), you'll learned how to use the methods in the Graphics class to draw an artist's cottage on the screen, and then use the Color class to paint it. This week we'll give your cottage an upgrade.

We'll start with a little variable review from Chapter 4. Most likely, when you created your Artist retreat, you used literals to specify the locations of the doors, windows and roof. That's really not a good idea for several reasons, so we'll "rewire" your house using variables.

Of course, even with variables, your house will end up a fixed size. It would be nicer if we could find out how big our applet window was, and just draw the house to fit right inside it. Java has several measuring classes that facilitate this. (You've already met one of them, the Rectangle class).

Finally, we'll finish up this lecture by introducing you to the Font and FontMetric classes that will allow you to control how and where your text is displayed.

Go ahead and open BlueJ and then open the Artist applet from last week.




## Slide 2

**Literals and Magic Numbers**
Duration: 00:00:60
Advance mode: Auto

## Notes:

Take a look at the code you've written for the Artist applet. How easy is it to read? If you didn't run it, could you tell that it was meant to draw a house? Probably not.

When you pass arguments (or parameters) to a method, you supply information that the method uses to carry out its, actions. When you write:

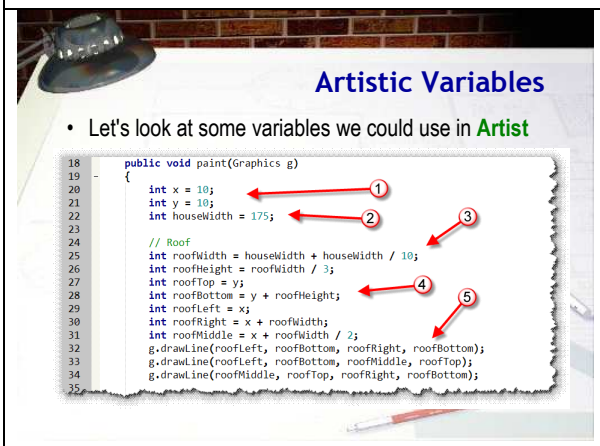
```
g.drawRect(   0,   0, 100, 100);
```

the drawRect() method uses the values you supply--0, and 100--to carry out the action of displaying a rectangle on the screen. The values passed in this example are called literals, because the value you use appears literally in your source code. Sometimes these are also called constants

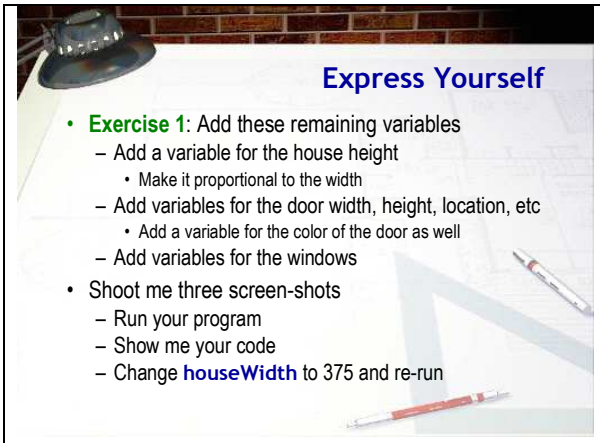
Unfortunately, using literal values like this is less than ideal, for two reasons:

- No meaning is associated with each number. A reader unfamiliar with the drawRect() method, can't easily tell the difference between the two uses of the literal 100 in the first line of code. For this reason, such literals are called magic numbers.
- If you want to make changes to a section of code, such as changing the width of each of the rectangles shown above to 95 pixels (from 100), using literals makes your work much more difficult and error-prone. You can't just change every 100 to 95, because other values may depend on the original 100.

| | |
|---|---|
| | So, what's the solution? Replace those literals with named values, that is, constants and variables. |
|  <br>**Slide 3**<br>**Artistic Variables**<br>Duration: 00:00:17<br>Advance mode: Auto | **Notes:**<br>Let's look at some variables we could use in our Artist applet to make it clearer and more flexible. Even though the program will actually be longer, it's kind of like when you're building a real house: doing it right usually pays off eventually.<br>The code I've started here (and that you'll complete), uses variables to instead of literals.<br>Instead of hard-coding an offset of 10 for the top and left positions of the roof, I start by creating a pair of variables, x and y, that I can reuse whenever I need that location. (Or, modify if I want to move the entire house around the screen.)<br>Next, I define one "master" variable, houseWidth that I set to 175. I'll use this variable whenever I define any of the other variables. After all, we want the roof to cover the house, right? And if we scale our drawing to twice size, or if we reduce it, we certainly want the windows and doors to follow.<br>You can see how this works in the roof section. Instead of specifying a literal roof overhang, I make the width of the roof the same as the width of the house with a 10% overhang. Then if you draw a larger house, the proportion stays correct. The same thing happens with the height of the roof; instead of hard-coding a roof height of 100, you can make it proportional to the roof width. I've added a pretty steep roof with a height that's 1/3 the width.<br>Once you've decided on the proportions, you can then define specific locations for your drawing. For instance, I've defined variables for the top, left, bottom, right and center of the roof.<br>Finally, you can use those variables when you draw the lines. Notice unlike the original code to draw the roof which didn't help you to visualize what was being drawn, the code now actually tells you where each line segment is going to go. |




## Slide 4

### Express Yourself

Duration: 00:01:05
Advance mode: Auto

## Notes:

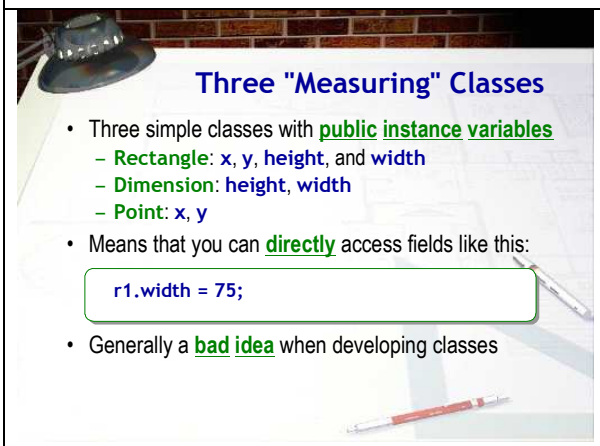
OK, let's see if you can put this to work. Create a new in-class Lab exercise document for this week. (See the class Web page for the exact name you need to use.) Start a new section and for Exercise 1, add these remaining variables.

- First, create a variable for the height of the house. Make sure your house height is proportional to the width of the house. That is, when we change the width, the height should vary appropriately. When you've done that, change the drawRect() that draws the house to use your new variables. Again, make sure you don't use any magic numbers.
- Next, do the same thing for the door. You'll need several variables to completely define the width, height and location of the door, as well as a separate variable for the door color. Change the setColor and fillRect calls to use your new variables.
- Finally, do the same thing for all of the windows. Create the appropriate variables for the dimensions as well as for the window color, and change the code that draws the windows to use your new variables.

Now, let's see how you've done. I want to see three screen-shots.

- First, run your program and show me a picture of that.
- Next, show me your code. (Actually that doesn't have to be a screen-shot; you can just copy and paste).
- Finally, (and this is the moment of truth), change houseWidth from 175 to 375 and re-run. If you've done it correctly, your house should be proportionally larger and all of the doors and windows should be the right sizes and in the right locations. If not, then modify your code until they are. Shoot me one final screen-shot.

By the way, don't worry about the text along the bottom of the screen. We'll see how to adjust that shortly.




## Slide 5

## Notes:

Before we finish drawing our cottage, let's take a look at three "helper" or "utility" classes that are used throughout the Java Class Libraries for different kinds of measurements: Rectangle, Dimension, and Point.

You've already met the Rectangle class, of course. In graphical programs, though, the Rectangle class is often used when talking about the size and position of a visual component, such as a button or a label, on the screen. For such objects, there are four relevant pieces of information:

- x : the number of pixels from the left-hand side of the component's container to the left-hand edge of the

## Three "Measuring" Classes

Duration: 00:00:48
Advance mode: Auto

component.

- y : The number of pixels from the top of the component's container to the upper edge of the component.
- width : The number of pixels across the component in a horizontal direction.
- height : The number of pixels across the component in a vertical dimension.

The Rectangle class bundles together these four pieces of information in one convenient package.

The Dimension class contains only the fields that have to do with the size of an object, height and width, and don't include information about the location. The Point class contains the location information, x and y, and not the size.

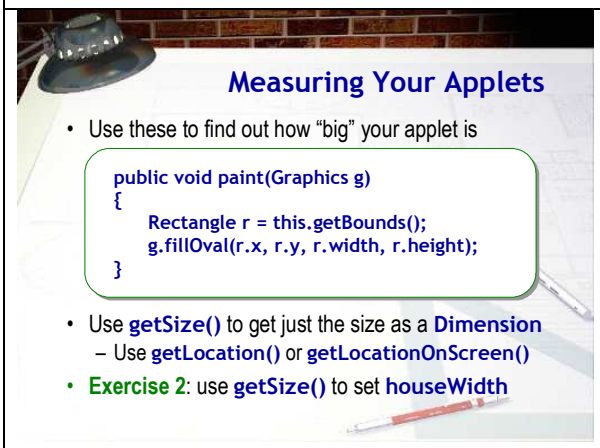
The fields inside the Rectangle, Dimension and Point classes also have one other unusual characteristic that's not found in most classes: instead of being private, which is the normal state for an instance variable, these fields are public. This means that you can directly access the fields inside the object r1, including changing the object just by assigning a new value, or reading the current value, like this:

```
r1.width = 75;
```

The designers of the Java class libraries, (folks like Joshua Bloch, for instance), have loudly said that this was a real mistake. In his book, Effective Java, on page 99 he writes:

"Several classes in the Java platform libraries violate the advice that public classes should not expose fields directly. Prominent examples include the Point and Dimension classes in the java.awt package. Rather than examples to be emulated, these classes should be regarded as cautionary tales.

The decision to expose the internals of the Dimension class resulted in a serious performance problem that could not be solved without affecting existing programs."




## Slide 6

### Measuring Your Applets

Duration: 00:00:36

## Notes:

Java's Component class makes use of the Rectangle class to store the current size and position of any of the objects on-screen. To retrieve the current size and position of any Component, you send it the getBounds() message and it gives you back a Rectangle object.

Since the Applet class is a descendant of Component, this works for your applets as well. You can find out how wide and high your applet is by calling its getBounds() method to retrieve a Rectangle object, and then using the fields in the Rectangle object, like this to fill the entire screen with an oval, no matter how high or wide it is.

```
public void paint(Graphics g)
{
```

Advance mode: Auto

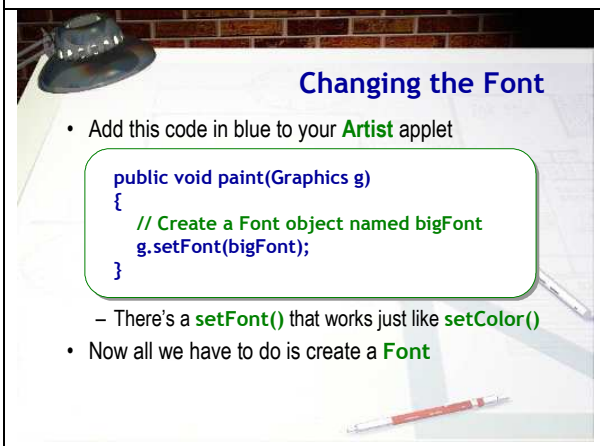
```
    Rectangle r = this.getBounds();
    g.fillOval(r.x, r.y, r.width, r.height);
}
```

Note that in the example here I've used the name this for the receiver, to make it explicit that I'm asking for the applet's bounds. You could also just write: getBounds() with no receiver.

The Component class also has methods that allow you to retrieve just the height and width of a Component, separately from its position on your applet. This method is getSize() and it returns a Dimension object.

Similarly, you can retrieve the Point where any Component is located by using getLocation() or getLocationOnScreen(). The getLocationOnScreen() method returns the position measured from the upper-left corner of your screen, while getLocation() returns the position measured from the upper-left corner of your applet itself.

For Exercise 2, let's put this to work. In your Artist applet, at the beginning of the paint() method, use getSize() to retrieve the size of the applet when it runs. Then, set the houseWidth variable using the 90% of the width field. Run your program and resize it by dragging the window. Shoot me a screen-shot.




## Slide 7

### Changing the Font

Duration: 00:00:30
Advance mode: Auto

## Notes:

Centered at the bottom of your Artist applet, you have the phrase "Steve's Dream House", with my name replaced with yours. Of course, since you hard-coded the position you used with drawString(), it probably doesn't stay centered if you change the size and font, or when you change the size of the window.

Let's look at changing the font first, and then we'll handle the positioning.

Go ahead and add this code to your Artist applet, right above where you display your picture's caption. Note that just as setColor() changes the default painting color used by the Graphics object g, setFont() causes the g to use a different font when it draws text.

The only problem is, as soon as you add this code, your program no longer compiles because bigFont doesn't yet exist. So, all we have to do is create it.


## Slide 8

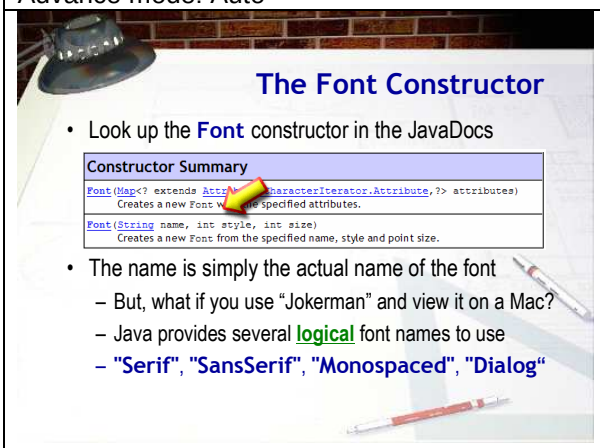
### Creating bigFont

Duration: 00:00:23
Advance mode: Auto

## Notes:

So, how do we create a Font object? Just like you create a Rectangle or Student or any other object. You use a constructor whose name is????

That's right, Font().




## Slide 9

### The Font Constructor

Duration: 00:00:36
Advance mode: Auto

## Notes:

Like most constructors, though, we'll need some additional information. Let's see what that is.

When we look up the Font class in the JavaDocs, we see that there are two constructors: One looks completely incomprehensible, (to me too!), but the other Font() constructor requires three arguments--name, style, and size--and each argument is something that we know how to supply, String and int parameters.

Let's look at the values we should supply.

The first argument to the Font constructor is a String that contains the actual name of the font. You can also specify a specific physical font, such as "MS Comic Sans", or "Jokerman", but, as with HTML, this only works if the requested font is located on the user's machine.

Instead, you should use one of these logical font names.

- "Serif"
- "SansSerif"
- "Monospaced"
- "Dialog"

As with HTML, you should normally use the platform-independent logical font names that will be replaced with Arial or Helvetica or New Times Roman, etc. depending upon the platform the application runs on, if you're going to distribute your program as an applet.


## Slide 10

### Font Style and Size

Duration: 00:00:59
Advance mode: Auto

## Notes:

The font style argument is an int, but you should always use one of the class constants that have been predefined inside the Font class. These work very much like the constants Color.red, etc. which are predefined inside the Color class.

The values you can use for this second argument are:

- Font.PLAIN, Font.BOLD or Font.ITALIC

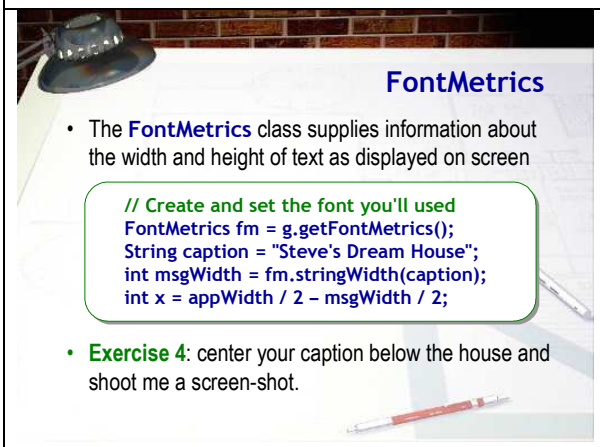
You can also use Font.BOLD + Font.ITALIC

Using something like Font.PLAIN + Font.BOLD doesn't make any sense, but it won't create a syntax error.

Make sure you get the capitalization right on these. Note that the words PLAIN and BOLD are ALL CAPS, while Font is capitalized, but also contains lowercase letters.

The last argument required by the constructor is the size of the Font. This is, theoretically, supplied as the point size for the font. Point size is a measurement used in printing. There are 72 points to an inch, and normal body text is usually 10 or 12 points. In fact, Java's Font class seems to use the short-cut of equating pixels and point size. That works OK when your monitor is set to 72 dpi, but if you use a higher resolution such as 96 or 120 dpi, which is pretty common now days, your fonts will seem a little too small.

OK, let's try this out. For Exercise 3, create the bigFont variable. Make the size proportional to houseWidth so that it changes as you change the size of your window. You can use any style and font family that you like.




## Slide 11

### FontMetrics

Duration: 00:00:48
Advance mode: Auto

## Notes:

Our Artist applet looks pretty good. The only problem we have is centering and positioning the caption. To do that, you'll make use of new class called FontMetrics. The FontMetrics class provides information you can use about the height and width of a line of text in pixels when it is displayed on the screen. You can use that information to position and center your message at the bottom of the screen.

Here's what you need to do to get it to work:

- First, create and set the font you'll use to draw your caption. Make sure you do both of these steps (create the font and call setFont() to make it the active font) before you create your FontMetrics object. Otherwise, the measurements you'll be using won't match those actually in effect.
- Next, create a FontMetrics variable (traditionally named fm) and initialize it by sending the Graphics object g a getFontMetrics() message. It will reply by creating a FontMetrics object and returning it to you.
- Next, store your caption in a String variable so that it can be measured. You could actually use literals here, but

| | |
|---|---|
| | then you'd need to type exactly the same thing again when you printed it.<br>• Now, use the FontMetrics method stringWidth() to retrieve the number of pixels required to paint your message. Pass your caption as the parameter, like this.<br>• *Now you know how wide your message is. On the previous slide you found out how wide your applet was. To center your message, just divide the application width by 2 (to get the center), and subtract half the width of your caption. That's the x value you'll use for drawString().<br>So, what about the y value. Well, I'll leave that part up to you. All you really need to know is how high your applet is. Of course you can't just use that for the y value because then any descenders would be off the bottom of the screen. Take a look at the documentation for the FontMetrics class and see if you can find a message that will provide you with that information.<br>For Exercise 4, and to finish up this lecture, center your caption below the house and then shoot me a screen-shot. |